

Architecture

Music Podcast Player

1 CORE ARCHITECTURE

The Music Podcast Player uses a layered service architecture:

- **Frontend:** Vue single-page application with routes for user interaction.
- **API Service:** Flask REST API for public-facing logic.
- **DB Service:** FastAPI internal service for user/playlist persistence logic.
- **Data Stores:** PostgreSQL (relational app data), DynamoDB (rate-limiting), S3-compatible object storage (editable page content).

2 ARCHITECTURE DIAGRAM

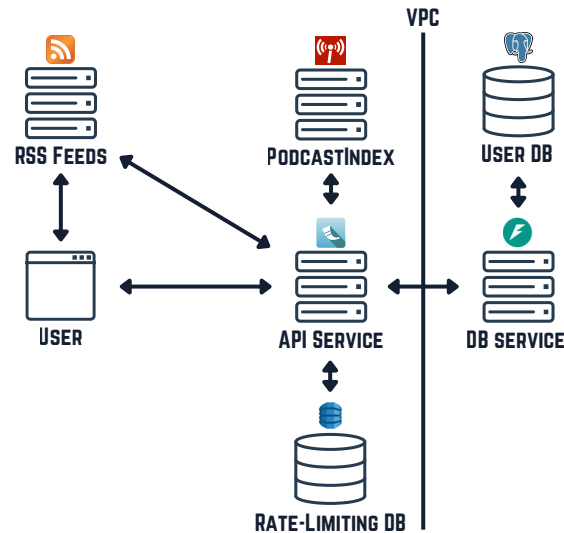


Figure 1: Software architecture: frontend, API layer, internal DB service, and data stores.

3 COMPONENT RESPONSIBILITIES

3.1 FRONTEND (VUE)

- Renders UI and manages user interaction.

- Calls backend endpoints for auth, search, feed parsing, playlists, and page content.
- Integrates Lightning wallet actions through Bitcoin Connect.

3.2 API SERVICE (FLASK)

- Public API boundary and request validation.
- Exposes route groups: `/auth`, `/search`, `/link`, `/playlists`, `/pages`.
- Applies authorization and rate-limit checks.
- Orchestrates calls to DB Service and external feed/search providers.

3.3 DB SERVICE (FASTAPI)

- Encapsulates persistence logic for users and playlists.
- Handles CRUD operations against relational data models.
- Returns typed responses to API Service.

3.4 STORAGE LAYER

- **PostgreSQL**: user accounts, playlists, playlist tracks.
- **DynamoDB**: token-bucket rate-limit counters and expiry state.
- **S3-compatible CMS storage**: editable HTML page payloads by page key.

4 PRIMARY DATA FLOWS

4.1 AUTHENTICATION FLOW

1. Frontend starts login flow and submits credentials.
2. API Service validates credentials via DB Service.
3. API Service issues access/refresh tokens and uses cookie-based session flow.
4. Frontend verifies/refreshes auth state through auth endpoints.

4.2 FEED SEARCH AND PARSING FLOW

1. Frontend sends search or feed URL request to API Service.
2. API Service validates request and applies rate-limit logic.
3. API Service calls external integrations for search/parsing.
4. Normalized feed data returns to frontend.

4.3 PLAYLIST FLOW

1. Authenticated user performs playlist action from frontend.
2. API Service authorizes and forwards persistence operation to DB Service.
3. DB Service updates PostgreSQL and returns result.
4. API Service may enrich track metadata before returning to frontend.

4.4 EDITABLE PAGE CONTENT FLOW

1. Public reads retrieve page content through `GET /pages/{page}`.
2. Admin edits submit sanitized HTML through `PUT /pages/{page}`.
3. API Service stores/retrieves page JSON in CMS object storage.

5 KEY DESIGN DECISIONS

- **Service separation:** public API logic is separated from persistence logic for maintainability and clearer boundaries.
- **Split backend boundary:** the backend is divided into an API Service and a DB Service. The DB Service is responsible for database access, while the API Service handles public requests and external integrations.
- **VPC isolation for database access:** database-connected components are placed inside the VPC so relational data access remains isolated from the public-facing application layer.
- **External API access outside the VPC:** the API Service is kept outside the VPC so it can communicate with external services such as the PodcastIndex API without requiring additional VPC egress infrastructure. It then communicates with the DB Service for persistence-related operations.
- **Hybrid storage:** relational store for core entities, key-value store for high-churn rate-limit state, object storage for editable static content.
- **Backend-mediated integrations:** external feed/search interactions are centralized in backend services for consistent validation and error handling.
- **Cookie-based auth:** simplifies frontend token handling and supports secure, session-like UX.

6 AUTHENTICATION ARCHITECTURE

The application uses cookie-based authentication with a PKCE-protected login flow. When the login page loads, the frontend generates a PKCE verifier/challenge pair. The verifier is stored in browser session storage, while the challenge is sent to the backend and stored in the server-side session. During login, the frontend submits the username, password, and verifier. The backend validates the credentials, recomputes the challenge from the verifier, and confirms that it matches the stored challenge before issuing authentication cookies.

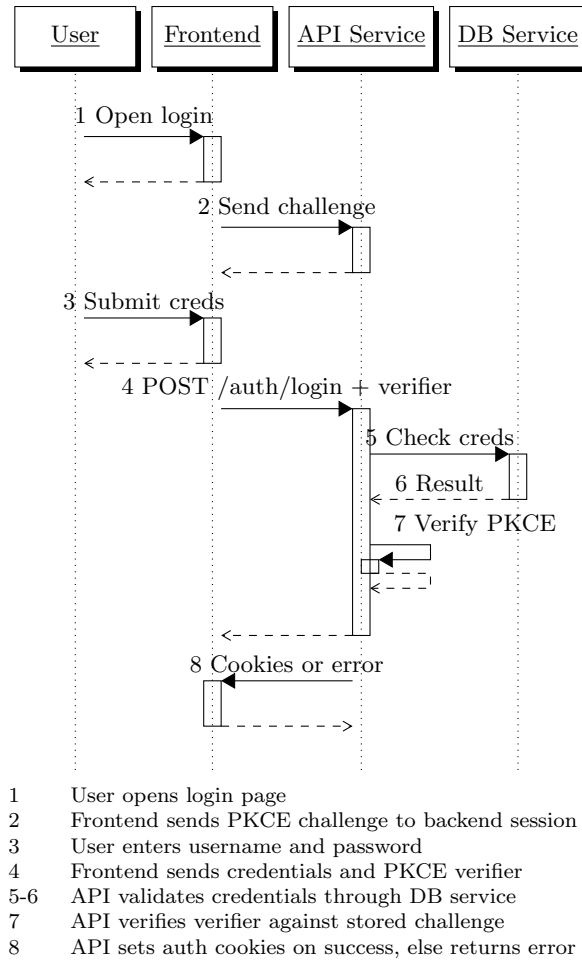


Figure 2: PKCE login flow

This design binds the login request to the browser session that initialized authentication and separates credential validation from token issuance.